PYMASTER INDIA · #1

Python Setup & Your First Program

Installation, VS Code, and How Python Works

Get a working Python setup on your machine from nothing.
Then understand what actually happens when you press Run.

Student Notes & Setup Guide
codeWithShivah · Rajeev Ranjan Pandey
PyMaster India Series

What You Will Learn

- Download and install Python 3.x (latest)
- Set up VS Code with the best extensions
- Understand how Python works — interpreter, .py files
- Write and run your very first Python program
- Fix the setup errors that stop most beginners

> **Start here.** This is the first video of PyMaster India — no prior coding experience needed. Everything in this series builds on the setup you do today, so take the time to get it right. Keep this PDF open while you follow along; the checklist at the end confirms whether your machine is genuinely ready.

1. Installing Python 3.x

Python is a program that reads and runs your code. Install it once, and your computer learns to understand the Python language.

Download

Go to **python.org → Downloads**. The site detects your operating system and offers the right installer. Take the latest stable Python 3 release. Never install Python 2 — it is retired and nothing in this series will work on it.

> **Windows — the one checkbox that matters.** On the first screen of the installer, tick **"Add python.exe to PATH"** at the bottom *before* clicking Install Now. Miss it and your terminal will not recognise the word `python`, which is the single most common setup failure for beginners. If you already missed it, re-run the installer, choose Modify, and add it.

macOS and Linux

Both ship with a system Python, but it is often old and belongs to the operating system. Install a current version yourself rather than relying on the built-in one:

```
# macOS — using Homebrew
brew install python

# Ubuntu / Debian Linux
sudo apt update
sudo apt install python3 python3-pip
```

Verify the install

Open a terminal — Command Prompt or PowerShell on Windows, Terminal on macOS and Linux — and type:

```
# Windows
python --version

# macOS / Linux
python3 --version

# you should see something like:
Python 3.13.1
```

If you see a version number, Python is installed and your terminal can find it. That is the whole test. If you see "*not recognized*" or "*command not found,*" jump to section 5 before going any further.

> **python vs python3.** On Windows the command is usually `python`. On macOS and Linux it is usually `python3`, because `python` may point to an old version 2. Find out which one works on your machine and use that throughout the series.

2. Setting Up VS Code

Python runs your code. **VS Code** is where you write it — a free editor from Microsoft that gives you colour-coded code, error highlighting, and a built-in terminal.

Install

Download from **code.visualstudio.com** and run the installer. On Windows, tick **"Add to PATH"** and the two **"Open with Code"** context-menu options — they let you right-click any folder and open it directly in the editor.

The extensions worth having

Open the Extensions panel with **Ctrl+Shift+X** (**Cmd+Shift+X** on macOS) and install:

Extension	What it does for you
Python (Microsoft)	The essential one — running code, IntelliSense, debugging, interpreter selection
Pylance	Fast, smart autocomplete and type checking; usually installs alongside Python
Code Runner	Run any file with one click or Ctrl+Alt+N
Black Formatter	Auto-formats your code to standard Python style on save
Material Icon Theme	Clear file-type icons in the sidebar — small thing, real quality-of-life gain

Point VS Code at your Python

Press **Ctrl+Shift+P**, type **Python: Select Interpreter**, and choose the version you just installed. VS Code needs to be told which Python to use — this step is easy to skip and causes confusing errors later.

> **Two settings worth turning on now.** Open Settings with **Ctrl+,** and enable **Format On Save**, plus **Auto Save** if you would rather not press Ctrl+S constantly. Your code stays tidy without you thinking about it.

3. How Python Actually Works

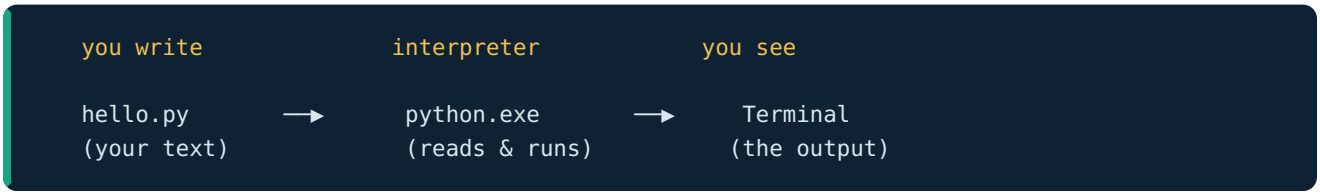
Understanding this now will save you confusion for the rest of the series.

Your code is just a text file

A Python program is a plain text file ending in **.py**. There is nothing special inside it — you could open it in Notepad. The `.py` extension simply signals "this file contains Python code."

The interpreter reads it line by line

When you run a file, the **Python interpreter** — the program you installed in section 1 — opens it and executes it from top to bottom, one line at a time.



This top-to-bottom, one-line-at-a-time behaviour explains a lot of what you will meet later — why order matters, why an error stops everything below it, and why a variable has to be created before it is used.

Compiled vs interpreted

	Interpreted (Python)	Compiled (C, Java)
Before running	Nothing — just run the file	A separate build step first
Speed to test a change	Instant	Rebuild, then run
Errors appear	When that line is reached	Many caught before running
Raw execution speed	Slower	Faster

This is the trade Python makes: you give up some raw speed and get a much faster write-test-fix cycle. For learning, for scripting, and for most data work, that is a straightforwardly good deal.

4. Your First Python Program

Make a home for your code

Create a folder — `pymaster` on your Desktop is fine — and open it in VS Code via **File → Open Folder**. Always open the *folder*, not a lone file; VS Code behaves better and everything later in the series assumes it.

Write it

In the sidebar, click **New File** and name it `hello.py`. The `.py` matters — that is what tells VS Code to treat it as Python.

```
# hello.py — my first Python program
print("Hello, World!")
print("My name is Amit")
print("I am learning Python with PyMaster India")
```

A few things to notice. `print()` displays whatever you put inside the brackets. Text goes in quotes — single or double, but be consistent. A `#` starts a comment, which Python ignores entirely; comments are notes for humans reading the code.

Run it

Method	How
Run button	The ▷ triangle at the top-right of the editor
Code Runner	**Ctrl+Alt+N**
Terminal (Windows)	`python hello.py`
Terminal (macOS/Linux)	`python3 hello.py`

```
$ python hello.py
Hello, World!
My name is Amit
I am learning Python with PyMaster India
```

Three lines of code, three lines of output, in the order you wrote them. That is the interpreter working exactly as section 3 described.

Break it on purpose

Errors are information, not failure — so meet your first one deliberately. Remove the closing quote and run again:

```
print("Hello, World!)

# SyntaxError: unterminated string literal (detected at line 1)
```

Read what Python told you: the *type* of problem (SyntaxError), a plain description, and the line number. Almost every error you meet in this series follows that same shape. Put the quote back and it runs again.

> **The interactive shell.** Type `python` (or `python3`) alone in the terminal and you get `>>>` — a live Python prompt where each line runs as you press Enter. Excellent for testing a quick idea. Type `exit()` to leave. Use it for experiments; use `.py` files for anything you want to keep.

5. When Setup Goes Wrong

Nearly every beginner hits at least one of these. None of them mean you did something wrong.

What you see	What to do
`'python' is not recognized...`	PATH was not set. Re-run the installer, choose Modify, tick **Add to PATH**. Restart the terminal afterwards.
`command not found: python` (macOS/ Linux)	Try `python3` instead — it is the correct command on those systems.
Microsoft Store opens instead of Python	A Windows placeholder is intercepting the command. Settings → Apps → Advanced app settings → App execution aliases → turn **off** both Python entries.
VS Code Run button does nothing	The Python extension is missing or no interpreter is selected. Install it, then Ctrl+Shift+P → Python: Select Interpreter.
Terminal still fails after installing	Close VS Code and the terminal completely and reopen. PATH changes only apply to newly opened windows.
`SyntaxError` on a line that looks fine	Check the line *above* it — an unclosed bracket or quote usually reports on the following line.
File runs but nothing prints	You likely saved as `hello.py.txt`. Check the real filename; enable file extensions in your file explorer.

> **Still stuck?** Copy the exact error text — not a description of it — and search it, or post it in the video comments. The precise wording is what makes a problem solvable.

Setup Checklist

There is no homework for this video — the work *is* the setup. Go through these six checks before starting #2. Every later video in the series assumes all six pass.

CONFIRM ALL SIX

CHECK 1

Python responds in the terminal

`python --version` (or `python3 --version`) prints a 3.x version number.

If not: section 5, first two rows.

CHECK 2

VS Code is installed with the Python extension

The Extensions panel shows Microsoft's Python extension as installed.

CHECK 3

An interpreter is selected

Ctrl+Shift+P → Python: Select Interpreter shows your 3.x version chosen. The version also appears in the VS Code status bar.

CHECK 4

hello.py runs

Your file prints all three lines, in order, with no errors.

CHECK 5

You have seen an error and fixed it

You removed a quote, read the SyntaxError, and put it back. You know what an error message looks like and where the line number appears.

CHECK 6

You have a project folder

A dedicated folder — `pymaster` or similar — that you open in VS Code and keep all series work inside.

OPTIONAL — TRY THESE

TRY 1

Make it yours

Edit `hello.py` to print your own name, your college, and one reason you are learning Python. Run it again.

TRY 2

Explore the shell

Open the interactive prompt and try `2 + 2`, then `print("hi" * 5)`, then `len("PyMaster")`. Note what each returns — you are meeting numbers, strings, and functions before we formally cover them.

TRY 3

Start the habit

Create a `videos/` folder inside your project with a subfolder per episode. Every video from here adds files — starting organised now saves a mess later.

Next up — Video #2. With your setup working, we start writing real Python: variables, data types, and input from the user. If any of the six checks above failed, fix it before moving on — everything after this point depends on a working environment. Drop a screenshot of your first program in the comments!